

After Action Report

Trio v5.0 RC2 Development Session

Date	2026-04-06T17:46Z
Duration	~1.25 hours
Channel	v49-test (397 messages, 101 in final segment)
Participants	Coordinator (Opus), Legolas (Opus, live) Gandalf, Sauron, Frodo, Aragorn (War Council)

OBJECTIVE

Ship v5.0 — a unified monitoring architecture for Trio multi-agent channels that dramatically reduces idle token cost.

WHAT SHIPPED

v4.9 — Agent-Based Idle Monitoring

- Background Agent loops the wait script internally, absorbing empty timeouts in ~10K context instead of cycling the parent's 200K+
- Empirically validated: agents survive 20+ loops, notify parents on completion, inherit Bash permissions
- 95% idle token reduction

v5.0 RC1 — Unified Sentinel

- Merged wait script (message detection) and watchdog (heartbeat/cadence) into a single sentinel script
- Single process, single DB connection, auto-detects active/idle/sleep mode from status_text
- status_changed_at column for transition tracking
- send() auto-clears sleeping keywords (server-side enforcement)
- Flag inconsistency detection (2-consecutive-observation threshold)
- Sleep confirmation (60s verified silence before relaxing thresholds)
- 84% total session token reduction

v5.0 RC2 — Dual-Sentinel Pattern

- Two parallel Haiku agents: message sentinel (returns on messages) + watchdog sentinel (returns on anomalies)
- Neither can die silently — if message sentinel dies, watchdog detects heartbeat staleness
- Internal looping: cap/error events handled inside the agent, never surface to parent
- "Relaunch FIRST, process SECOND" rule
- Emergency protocol: when watchdog fires, relaunch immediately, check message sentinel
- Shared SLEEPING_KEYWORDS constant extracted to prevent drift
- idx_messages_channel_member index for sentinel query performance

KEY DISCOVERIES

The Frodo Problem (Revisited)

The developer of the cadence nag system forgot their own cadence during testing. This led to the "relaunch FIRST" rule and the dual-sentinel architecture. The insight: behavioral instructions get forgotten under cognitive load. Mechanical enforcement (sentinels watching each other) is the only reliable solution.

Agent-as-Monitor Pattern

Background agents can loop internally on timeouts, absorbing empty cycles in their own ~10K context instead of cycling the parent's 200K+. This is the foundational insight. Everything else — the sentinel, the dual pattern, the mode detection — builds on this.

Bash Sleep as Free Infrastructure

`sleep N && echo "payload" via run_in_background=true` costs zero LLM tokens. Useful as dumb nag timers. Superseded by the sentinel for production use but validated as a pattern.

Flag Inconsistency is Self-Correcting

The sentinel's flag inconsistency detection can only fire during quiet channels because `new_messages` events preempt it. This is correct behavior — during active channels, message detection wakes the parent anyway.

send() Auto-Clear: Invisible Mode Transition

When an idle agent responds to a message, `send()` silently clears their sleeping status. The agent is now in active mode (3s sentinel checks, cadence enforcement) without being told. This interaction was undocumented until Frodo flagged it in the War Council.

WAR COUNCIL REVIEW

Full council deployed against RC2: Sauron (correctness), Gandalf (architecture), Frodo (UX), Aragorn (security), Legolas (performance, live on channel).

Reviewer	Criticals	Warnings	Notes
Sauron	0	4	5
Gandalf	1	5	4
Frodo	2	5	4
Aragorn	0	1	7
Legolas	0	2	0
Total	3	17	20

All 3 criticals resolved (SKILL.md contradictions from stale RC1 text). Top warnings addressed: shared keyword constant, emergency protocol accuracy, cadence threshold documentation, prompt clarity for Haiku agents.

TOKEN ECONOMICS

8-Hour Session: 1 Worker + 2 Idle Helpers

	Opus Tokens	Haiku Tokens	Total
v4.8	38.2M	0	38.2M
v5.0 RC2	15.1M	800K	15.9M
Reduction	60%		58%

Monitoring overhead went from 63% of total session cost to 5%. The worker's actual dev work is now the dominant cost.

5-Hour Idle Monitoring Only

	Tokens	Dollar Cost
v4.8 (Opus parent)	9M Opus	\$135
v5.0 RC2 (Haiku sentinels)	42K Haiku	\$0.03
Reduction		99.98%

WHAT WE DIDN'T GET TO

- **Sonnet triage layer (v5.1)** — a Sonnet agent between sentinel and parent that filters channel noise. Branch exists, TODO filed.
- **Long-duration soak test** — 2+ hours sustained sentinel monitoring in production
- **Sleep confirmation end-to-end** — requires real 60s+ silence to validate

ARCHITECTURAL DECISIONS

1. **Server stays protocol-agnostic** — monitoring strategy lives in SKILL.md and the sentinel, not in MCP tool responses.
2. **Sentinel is a Python script, not an MCP tool** — runs inside a Bash call inside a Haiku agent. No MCP registration changes needed.
3. **Dual sentinels over single sentinel** — the Frodo Problem (forgetting to relaunch) is the dominant failure mode. Two sentinels watching each other is cheap insurance.
4. **Auto-clear on send() is belt; flag inconsistency is suspenders** — both exist because neither alone is sufficient.
5. **10-minute watchdog cadence threshold** — calibrated through live testing. 60s was too aggressive. 120s still triggered during builds. 600s is "genuinely dead" territory.

PROCESS NOTES

- The entire v4.9 → v5.0 RC2 evolution happened in one continuous session with live testing on a trio channel
- Legolas served dual roles: live channel participant AND War Council performance reviewer
- Every feature was validated empirically before committing, not just designed
- The War Council review caught 3 criticals that would have confused cold-start agents — all text issues in SKILL.md, not code bugs
- 10 commits total, each capturing a meaningful step in the evolution